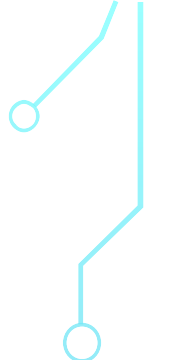
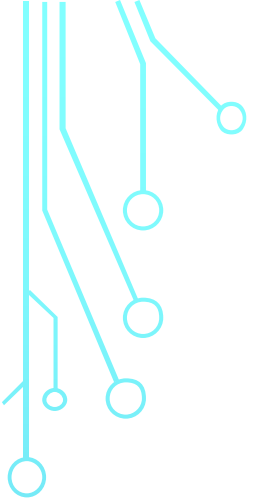




Arrays

I/O Redirection

Strings



Reminder

Make sure you have a TA verify your setup from HW0 by Thursday.

Arrays

Arrays

Let's write a program that counts the number of occurrences of each digit.

```
int main() {  
    int c;  
    int ndigit[10] = {0};  
}
```

↑
An array to hold the
count of each digit.

← C does not
automatically zero-
initialize arrays, so
we must explicitly
set each value to 0.

Arrays

Let's write a program that counts the number of occurrences of each digit.

```
int main() {  
    int c;  
    int ndigit[10] = {0};  
}
```

A Magic Number! Oh No!

Magic Numbers

- A **magic number** is a numeric value written directly in code without a clear explanation of what the number represents.
- We can use `#define` to give the value a meaningful name, making the program easier to read and maintain.

Arrays

```
#define NUM_DIGITS 10
```

```
int main() {  
    int c;  
    int ndigit[NUM_DIGITS] = {0};  
}
```

#define

- **#define** is a **preprocessor directive** in C used to create constants or macros.
- It is handled by the **C preprocessor**, before the actual compilation begins.
- Basic Syntax
 - **#define NAME value**
 - NAME = identifier you create
 - value = text that will replace NAME in the program

C Preprocessor

- The **preprocessor** is a program that prepares your C source code for the compiler. It looks for instructions beginning with #, called **preprocessor directives**.

- For example:

```
#define MAX_STUDENTS 30  
int classSize = MAX_STUDENTS;
```

- The preprocessor replaces **MAX_STUDENTS** with **30** before the compiler processes the program. Conceptually, the compiler receives something like:

```
int classSize = 30;
```

#define vs Variables

Feature	#define	Variable
Memory allocated	No	Yes
Type checking	No	Yes
Can change value	No	Yes
Preprocessor replacement	Yes	No

Build Process

Stage	What it does
1. Preprocessor	Handles #include, #define, and other preprocessor directives
2. Compiler	Translates the resulting C code into assembly language
3. Assembler	Translates assembly instructions into machine code
4. Linker (we will cover this step later)	Combines your machine code with libraries and other object files to create the final executable

Full ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Arrays

Let's write a program that counts the number of occurrences of each digit.

```
#define NUM_DIGITS 10

int main() {
    int c;
    int ndigit[NUM_DIGITS] = {0};

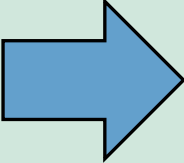
    while ((c = getchar()) != EOF) {
        if (c >= '0' && c <= '9') {
            // increment the count of the current character
        }
    }
}
```

Checking to see if the input is a digit.

An arrow points from the text "Checking to see if the input is a digit." to the condition `c >= '0' && c <= '9'` in the `if` statement of the code.

Arrays

How do we increment the count of the current character?



```
#define NUM_DIGITS 10

int main() {
    int c;
    int ndigit[NUM_DIGITS] = {0};

    while ((c = getchar()) != EOF) {
        if (c >= '0' && c <= '9') {
            // increment the count of the current character
        }
    }
}
```

```
ndigit[c]++;
ndigit['c' - '0']++;
ndigit[c - '0']++;
ndigit[c - 48]++;
```

Arrays



How do we increment the count of the current character?

```
#define NUM_DIGITS 10

int main() {
    int c;
    int ndigit[NUM_DIGITS] = {0};

    while ((c = getchar()) != EOF) {
        if (c >= '0' && c <= '9') {
            ++ndigit[c - '0'];
        }
    }
}
```

Arrays



- `ndigit[c]++;`
 - This directly uses the character `c` as the array index. Since `c` represents the ASCII value of the input character, the index will be between 48 and 57 for digits ('0' to '9'). `ndigit` only has 10 elements (indices 0-9), so this will index out of bounds.
- `ndigit['c' - '0']++;`
 - The expression `'c' - '0'` is incorrect because `'c'` is treated as a literal character, not the value of the variable `c`. `'c' - '0' = 99 - 48 = 51`
- `ndigit[c - '0']++;`
 - This is the correct approach. Subtracting `'0'` from the character `c` converts the ASCII value of the digit (e.g., 48 for '0', 49 for '1') to its numeric equivalent (e.g., 0 for '0', 1 for '1').
- `ndigit[c - 48]++;`
 - Subtracting 48 (the ASCII value of '0') from `c` is functionally equivalent to subtracting `'0'`. This will correctly map the ASCII value of the digit to its numeric equivalent.
 - While this works, it is less readable. Using `'0'` instead of 48 makes the intent clear.

Array Size

- C arrays

- Just blocks of contiguous memory
- Do not store their own size
- Programmer must track and pass size separately

```
void printArray(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
}
```

- Java arrays

- Are objects with built-in metadata
- Always know their own length (.length)
- No need to manage size manually

```
void printArray(int[] arr) {  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");  
    }  
}
```

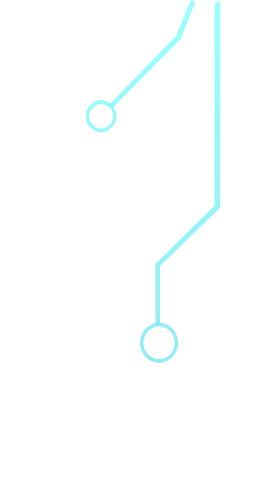
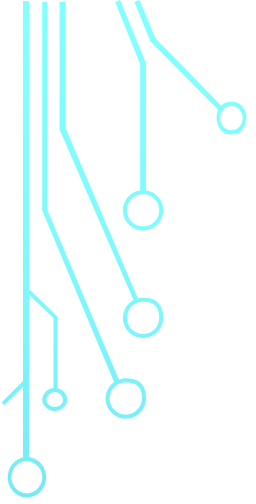
I/O Redirection

I/O Redirection

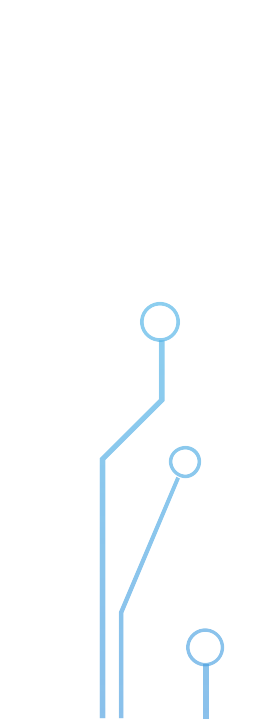
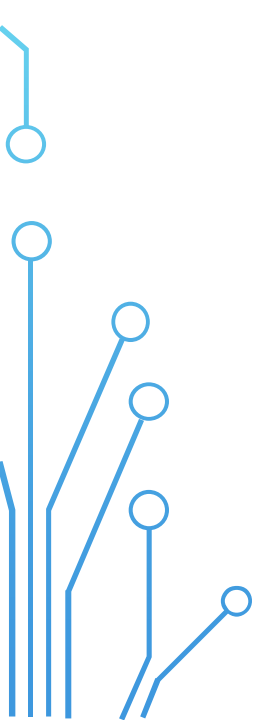
- A **file descriptor** is a non-negative integer that uniquely identifies an open file within a process.
 - The operating system uses file descriptors to manage input and output resources such as files, devices, and network connections.
- A **file stream** is a higher-level abstraction built on top of file descriptors. It provides buffered input and output operations through standard library functions.
- Every process begins with three open file streams by default:
 - **Standard input (stdin)**: the source from which a process reads its input
 - **Standard output (stdout)**: the default destination for normal program output
 - **Standard error (stderr)**: the destination used for error messages and diagnostics

I/O Redirection

- When a new process starts, these three streams are automatically opened and associated with the following file descriptors:
 - **0: stdin**
 - **1: stdout**
 - **2: stderr**
- These default streams allow programs to communicate with the user and the system without needing to explicitly open any files.



Standard In (stdin)



stdin

- By default, **stdin** reads input from the terminal (keyboard)
- You can change where **stdin** comes from (redirection)

- Syntax:

```
command < input_file
```

- Meaning:
 - Program reads input from a file instead of the keyboard

Why stdin Redirection Is Useful

- Automates program input
 - Avoids manual typing
- Allows programs to work together
- Makes programs reusable in scripts
- Example uses:
 - Feeding test data to programs
 - Processing large files
 - Running batch jobs

Demo Program: readstdin.c

```
#include <stdio.h>

int main(void) {
    int c;
    printf("Reading from stdin...\n");
    while ((c = getchar()) != EOF) {
        putchar(c);
    }
    printf("\nReached EOF.\n");
    return 0;
}
```

c is an int, not a char, because getchar() needs to be able to return every possible character **plus the special value EOF**.

Demo 1: Reading from the Terminal

- Step 1: Compile the program

```
gcc readstdin.c -o readstdin
```

- Step 2: Run the program

```
./readstdin
```

- Step 3: Then type an input. For example:

```
hello
```

```
this came from my keyboard
```

- The program reads each character with `getchar()` and prints it with `putchar()`.
- To tell the program that stdin has ended, press **Ctrl+D** on Linux/macOS. On Windows Command Prompt, use **Ctrl+Z**, then Enter.

Demo 2: Redirecting stdin

- Step 1. Create a file:

```
echo "Hello from a file!" > input.txt
```

- Step 2: Now run:

```
./readstdin < input.txt
```

- You'll get something like:

```
Reading from stdin...
```

```
Hello from a file!
```

```
Reached EOF.
```

- The important part is that **the C program did not change at all**. `getchar()` is still reading stdin.

Standard Out

Default Behavior of stdout

- By default, **stdout** writes to the terminal
- You can change where **stdout** goes (redirection)
- Syntax:

```
command > output_file
```

- This will overwrite output_file

```
command >> output_file
```

- This will append to output_file

Why Redirect stdout?

- Save program output to files
- Avoid cluttering the terminal
- Enable automation and scripting

Demo Program

```
#include <stdio.h>

int main(void) {
    char message[] = "Hello from stdout!\n";

    for (int i = 0; message[i] != '\0'; i++) {
        putchar(message[i]);
    }

    return 0;
}
```

Demo 1: stdout goes to the terminal

- Normally:

```
./writestdout
```

- You'll see:

```
Hello from stdout!
```

- So `putchar ( )` doesn't really mean "print this character on the screen." It means **write this character to stdout**.
- By default, your shell connects `stdout` to the terminal.

Demo 2: Redirect stdout to a file

- Now run:

```
./writestdout > output.txt
```

- Nothing appears on the terminal.
- Instead:

```
cat output.txt
```

shows:

```
Hello from stdout!
```

- The shell has changed where stdout points
 - Instead of pointing to the terminal, it points to output . txt

Append vs. Overwrite

- There's also an important distinction between:

```
./writestdout > output.txt
```

and:

```
./writestdout >> output.txt
```

- > replaces/truncates the file before writing, while >> appends to the end.

Append vs. Overwrite Example

```
./writestdout > output.txt
```

```
./writestdout >> output.txt
```

```
./writestdout >> output.txt
```

```
cat output.txt
```

produces:

```
Hello from stdout!
```

```
Hello from stdout!
```

```
Hello from stdout!
```

stderr

stderr

- stderr stands for **standard error**
- Used for:
 - Error messages
 - Warnings
 - Diagnostics

Demo

```
#include <stdio.h>

int main(void) {
    char message[] = "Error: something went wrong!\n";
    for (int i = 0; message[i] != '\0'; i++) {
        fputc(message[i], stderr);
    }

    return 0;
}
```

fputc

- Unlike `putchar()`, which always writes to `stdout`:

```
putchar('A');
```

- `fputc()` lets us specify the stream:

```
fputc('A', stdout);
```

```
fputc('A', stderr);
```

Demo 1

```
gcc writestderr.c -o writestderr
```

```
./writestderr
```

You'll see:

```
Error: something went wrong!
```

By default, `stderr` is connected to the terminal

Demo 2

- The shell uses file descriptor 2 for stderr, so you can redirect it with 2>:

```
./writestderr 2> error.txt
```

- Now the error doesn't appear on the terminal.

- Instead:

```
cat error.txt
```

gives:

```
Error: something went wrong!
```

Why 2> instead of >?

- The three standard streams correspond to three standard file descriptors:

Stream	File descriptor	Default destination
stdin	0	Terminal
stdout	1	Terminal
stderr	2	Terminal

- So:

```
./program > output.txt
```

is essentially shorthand for:

```
./program 1> output.txt
```

- But stderr requires:

```
./program 2> error.txt
```

Demo: Seeing the difference between stdout and stderr

```
#include <stdio.h>

int main(void) {
    char output[] = "This is normal output.\n";
    char error[] = "This is an error message.\n";

    for (int i = 0; output[i] != '\0'; i++) {
        fputc(output[i], stdout);
    }

    for (int i = 0; error[i] != '\0'; i++) {
        fputc(error[i], stderr);
    }

    return 0;
}
```

Demo 1

```
./program
```

- both appear on your terminal:

```
This is normal output.
```

```
This is an error message.
```

```
./program > output.txt
```

- Your terminal still shows:

```
This is an error message.
```

while output.txt contains:

```
This is normal output.
```

- That's because > only redirected stdout:

Demo 2

- You could separate them completely:

```
./program > output.txt 2> error.txt
```

- This separation is the main reason stderr exists: **normal program output can go one place while diagnostic/error messages go somewhere else.**

Active Learning

```
#include <stdio.h>

int main(void) {
    int c;
    while ((c = getchar()) != EOF) {
        putchar(c);
    }
    fprintf(stderr, "Done!\n");
    return 0;
}
```

1. What does `getchar()` read from?
2. Where does `putchar()` send "Hello!"?
3. Where does `fprintf(stderr, "Done!\n")` send its output?
4. What is inside `output.txt` after the program finishes?
5. What is inside `error.txt` after the program finishes?
6. What, if anything, appears on the terminal?

Suppose `input.txt` contains:

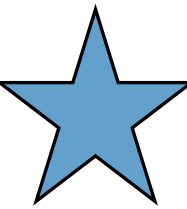
Hello!

You run:

```
./program < input.txt > output.txt 2> error.txt
```

`fprintf()` is like `printf()`, except you explicitly tell it which output stream to write to.

Active Learning



- **What does getchar() read from?**
 - It reads from stdin. Because of < input.txt, the shell has connected stdin to **input.txt**.
- **Where does putchar() send "Hello!"?**
 - putchar() writes to stdout. Because of > output.txt, stdout is connected to **output.txt**.
- **Where does fprintf(stderr, "Done!\n") go?**
 - It writes to stderr. Because of 2> error.txt, stderr is connected to **error.txt**.
- **What is inside output.txt?**
 - Hello!
- **What is inside error.txt?**
 - Done!
- **What appears on the terminal?**
 - **Nothing**, because all three standard streams have been redirected away from the terminal.

Pipes

Pipes

- A pipe connects the output of one command to the input of another
- It redirects:
 - `stdout` of the first command
 - to `stdin` of the second command
- Syntax

```
command1 | command2
```

- command1 writes to `stdout`
- command2 reads from `stdin`

Why Pipes are Powerful

- Avoid intermediate files
- Encourage small, reusable commands
- Enable complex tasks with simple tools
- Pipes are heavily used in:
 - Shell scripts
 - Log processing
 - Data analysis

Demo: producer.c

```
#include <stdio.h>

int main(void) {
    char message[] = "Hello through the pipe!\n";
    for (int i = 0; message[i] != '\0'; i++) {
        putchar(message[i]);
    }
    return 0;
}
```

Compile it:

```
gcc producer.c -o producer
```

Normally:

```
./producer
```

gives:

```
Hello through the pipe!
```

because its stdout goes to the terminal.

Demo: consumer.c

```
#include <stdio.h>

int main(void) {
    int c;
    while ((c = getchar()) != EOF) {
        putchar(toupper(c));
    }
    return 0;
}
```

Compile it:

```
gcc consumer.c -o consumer
```

Normally:

```
./consumer
```

waits for you to type because its stdin is connected to the terminal.

Connecting Producer and Consumer with a Pipe

- Run:

```
./producer | ./consumer
```

- produces:

```
HELLO THROUGH THE PIPE!
```

- The **|** tells the shell to connect
 - the **first program's stdout**
 - to the **second program's stdin**

Pipes Can be Chained

- You can have several programs connected together:

```
./producer | ./consumer | wc -c
```

- **wc** is a standard Unix/Linux command whose name means **word count**.
- It reads text from a file or from stdin and counts things such as lines, words, and bytes.
- You can ask for a specific count:

```
wc -l file.txt      # count lines
```

```
wc -w file.txt      # count words
```

```
wc -c file.txt      # count bytes
```

Active Learning

producer.c

```
#include <stdio.h>

int main(void) {
    printf("apple\n");
    printf("banana\n");
    printf("cherry\n");

    return 0;
}
```

consumer.c

```
#include <stdio.h>

int main(void) {
    int c;
    while ((c = getchar()) != EOF) {
        if (c == 'a' || c == 'e'
            || c == 'i' || c == 'o' || c == 'u') {
            putchar('-');
        } else {
            putchar(c);
        }
    }
    return 0;
}
```

Active Learning

Compile:

```
gcc producer.c -o producer  
gcc consumer.c -o consumer
```

Then run:

```
./producer | ./consumer > result.txt
```

1. What text does producer write to its stdout?
2. Where is producer's stdout connected?
3. What happens to the `\n` characters?
4. What exact output appears on the terminal?
5. What does `result.txt` contain after the command is run?

Active Learning



1. What text does producer write to its stdout?

apple

banana

cherry

2. Where is producer's stdout connected?

- To the stdin of consumer.

4. What happens to the `\n` characters?

- They are written unchanged, so the line breaks are preserved.

5. What exact output appears on the terminal?

- Nothing, because consumer's stdout is redirected into result.txt.

6. What does result.txt contain after the command is run?

-ppl-

b-n-n-

ch-rry

Strings

String Reading Summary

- In C, strings are implemented as arrays of char values
- Every C string must end with a null terminator (`'\0'`)
- The null terminator marks the end of the string in memory
- C provides a standard library for working with strings
 - Included with `#include <string.h>`
- String-handling functions in this library rely on the null terminator to determine where the string ends
- When printing a string using `printf`, use the `%s` format specifier

strlen vs sizeof

- The length of a string is not necessarily the same as the size of the array that stores it.
- `strlen` is used to count the number of characters in a string (excluding the null terminator)
- `sizeof` is used to determine the total size of the array (in bytes)

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char my_str[5];
    my_str[0] = 'h';
    my_str[1] = 'i';
    my_str[2] = '\0';
    printf("strlen(my_str) = %lu,
           sizeof(my_str) = %lu\n",
           strlen(my_str), sizeof(my_str));
}
```

```
learncli$ gcc temp3.c
learncli$ ./a.out
strlen(my_str) = 2, sizeof(my_str) = 5
```

Double Quotes vs Array

- When you create a string using double quotes (" "), the compiler automatically adds a null terminator to the end.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char my_str1[] = {'h', 'i', '\0'};
    char my_str2[] = "hi";
    printf("strlen(my_str1) = %lu, sizeof(my_str1) = %lu\n",
          strlen(my_str1), sizeof(my_str1));
    printf("strlen(my_str2) = %lu, sizeof(my_str2) = %lu\n",
          strlen(my_str2), sizeof(my_str2));
}
```

```
learncli$ gcc temp3.c
learncli$ ./a.out
strlen(my_str1) = 2, sizeof(my_str1) = 3
strlen(my_str2) = 2, sizeof(my_str2) = 3
```

```

#include <stdio.h>
#include <string.h>

int main(void) {
    /* 1. " " automatically includes a null terminator */
    /* A sequence of characters enclosed in double quotes ( " ") is
    called a string literal. */

    char str1[] = "hello"; // {'h','e','l','l','o','\0'}

    /* 2. ' ' does NOT include a null terminator */
    char str2[5] = {'h', 'e', 'l', 'l', 'o'}; // NOT a valid C string

    /* 3. Character array larger than the string - totally fine */
    char str3[10] = "hi"; // only uses 3 chars: 'h','i','\0'

    /* 4. Null terminator placed early */
    char str4[10] = {'h', 'i', '\0', 'X', 'Y', 'Z', '\0'};

    /* Print strings */
    printf("str1: \"%s\" (strlen = %lu)\n", str1, strlen(str1));
    /* WARNING: Uncommenting this causes undefined behavior */
    // printf("str2: \"%s\" \n", str2);
    printf("str3: \"%s\" (strlen = %lu)\n", str3, strlen(str3));
    printf("str4: \"%s\" (strlen = %lu)\n", str4, strlen(str4));

    /* 5. Demonstrating array size vs string length */
    printf("Array size of str3: %lu\n", sizeof(str3));
    printf("String length of str3: %lu\n", strlen(str3));
    return 0;
}

```